

Homework 02

680011465

2021-03-12T15:00:00 GMT

Contents

0. Data	2
1. Fit a Random Forest model	2
2. Compute and Compare Predictive Performance	3
3. Examine the model	7
Bonus: Use IML to create a PDP+ICE chart	10
4. Summarize	11

0. Data

We're going to use a mail response data set from a real direct marketing campaign located in `mailing.csv`. Each record represents an individual who was targeted with a direct marketing offer. The offer was a solicitation to make a charitable donation. This data was provided by the authors of our textbook, and I'm not sure of the original source.

The columns (features) are:

<code>income</code>	household income
<code>Firstdate</code>	data assoc. with the first gift by this individual
<code>Lastdate</code>	data associated with the most recent gift
<code>Amount</code>	average amount by this individual over all periods
<code>rfaf2</code>	frequency code
<code>rfaa2</code>	donation amount code
<code>pepstrfl</code>	flag indicating a star donator
<code>glast</code>	amount of last gift
<code>gavr</code>	amount of average gift
<code>class</code>	outcome variable, 1 if they gave donation

The target variable is `class` and is equal to one if they gave in this campaign and zero otherwise.

```
load('./mailing_train_test.RData')
glimpse(mailing_train)
```

```
## Rows: 4,000
## Columns: 14
## $ Income      <dbl> 7, 2, 4, 6, 6, 3, 1, 5, 2, 0, 0, 0, 6, 3, 6, 0, 5, 6, 7,...
## $ Firstdate   <dbl> 9101, 9501, 9410, 9110, 9510, 8610, 8612, 8809, 8703, 94...
## $ Lastdate    <dbl> 9512, 9508, 9602, 9512, 9511, 9601, 9701, 9601, 9511, 95...
## $ Amount      <dbl> 0.30, 0.23, 0.24, 0.17, 0.11, 0.45, 0.43, 0.23, 0.25, 0....
## $ rfaf2       <dbl> 3, 2, 4, 2, 2, 3, 1, 4, 2, 1, 1, 2, 1, 1, 4, 1, 2, 1, 3,...
## $ glast       <dbl> 10, 20, 8, 20, 15, 5, 5, 5, 5, 15, 15, 10, 18, 25, 10, 1...
## $ gavr        <dbl> 6.41, 15.00, 5.42, 11.50, 14.00, 3.29, 6.87, 6.56, 11.78...
## $ class       <fct> 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,...
## $ rfaa2_D     <int> 0, 0, 1, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,...
## $ rfaa2_E     <int> 1, 0, 0, 0, 0, 0, 1, 1, 0, 0, 0, 1, 0, 0, 1, 0, 0, 0,...
## $ rfaa2_F     <int> 0, 1, 0, 1, 1, 0, 0, 0, 1, 1, 1, 0, 1, 0, 0, 1, 1, 1,...
## $ rfaa2_G     <int> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0,...
## $ pepstrfl_0  <int> 0, 1, 1, 0, 0, 0, 0, 0, 0, 1, 1, 0, 0, 1, 0, 0, 1, 1, 0,...
## $ pepstrfl_X  <int> 1, 0, 0, 1, 1, 1, 1, 1, 1, 0, 0, 1, 1, 0, 1, 1, 0, 0, 1,...
```

1. Fit a Random Forest model

Our outcome variable is `class`. Provide a quick look at the distribution of `class` for the training data and test data.

```
summary(mailing_train$class)
```

```
##      0      1
## 2000 2000
```

```
summary(mailing_test$class)
```

```
##    0    1  
## 947  53
```

Comparing the distribution of the outcome variable in training and test, do they look balanced? >**According to the mailing_train data, it shows that 2000 individuals donate and 2000 individuals did not donate. Thus, this data is balance. >According to the mailing_test data, it shows that 53 individuals donate and 947 individuals did not donate. Thus, this data is not balance.**

Fit a random forest model to predict class.

```
rf <- randomForest(class ~., data = mailing_train, ntree = 1000, mtry = 5)
```

Examine the results.

```
print(rf)
```

```
##  
## Call:  
## randomForest(formula = class ~ ., data = mailing_train, ntree = 1000,      mtry = 5)  
##           Type of random forest: classification  
##           Number of trees: 1000  
## No. of variables tried at each split: 5  
##  
##           OOB estimate of  error rate: 42.02%  
## Confusion matrix:  
##           0    1 class.error  
## 0 1136  864      0.4320  
## 1  817 1183      0.4085
```

What is the out-of-box error rate? >**The default number of tree is 500 and the default number of variables tried at each split is 3. However, I have decided to go further by increasing the amount of trees to 1000 and the number of variables tried at each split to 5. The out-of-box/out-of-bag estimate error rate is 42.02%. This estimate error is decent but not optimal.**

2. Compute and Compare Predictive Performance

Use the `confusionMatrix` function to compute several metrics of predictive performance.

```
rf$confusion[, -3] %>%  
  as.table %>%  
  confusionMatrix()
```

```
## Confusion Matrix and Statistics  
##  
##           0    1  
## 0 1136  864  
## 1  817 1183  
##
```

```
##           Accuracy : 0.5798
##           95% CI  : (0.5643, 0.5951)
##    No Information Rate : 0.5118
##    P-Value [Acc > NIR] : <2e-16
##
##           Kappa : 0.1595
##
##  McNemar's Test P-Value : 0.2619
##
##           Sensitivity : 0.5817
##           Specificity : 0.5779
##    Pos Pred Value : 0.5680
##    Neg Pred Value : 0.5915
##           Prevalence : 0.4883
##    Detection Rate : 0.2840
##    Detection Prevalence : 0.5000
##    Balanced Accuracy : 0.5798
##
##    'Positive' Class : 0
##
```

How would you describe the performance of this model? > **The accuracy of the model is 57.98% with a 95% confidence interval bounded at 56.43% and 59.51%. Thus, I would say the accuracy is quite decent. The value for sensitivity is 58.17%, while the value for specificity is 57.79%; therefore the data is more sensitive than specific. Furthermore, the data is slightly better at predicting people who donate than people who do not donate.**

The confusion matrix assumed a 0.50 cutoff for prediction. Now create a ROC plot and compute the AUC for the training set.

```
p <-predict(rf, newdata = mailing_train, type = "prob") [,2] %>%
  as.numeric
mailing_train_roc <- roc(mailing_train$class, p)
```

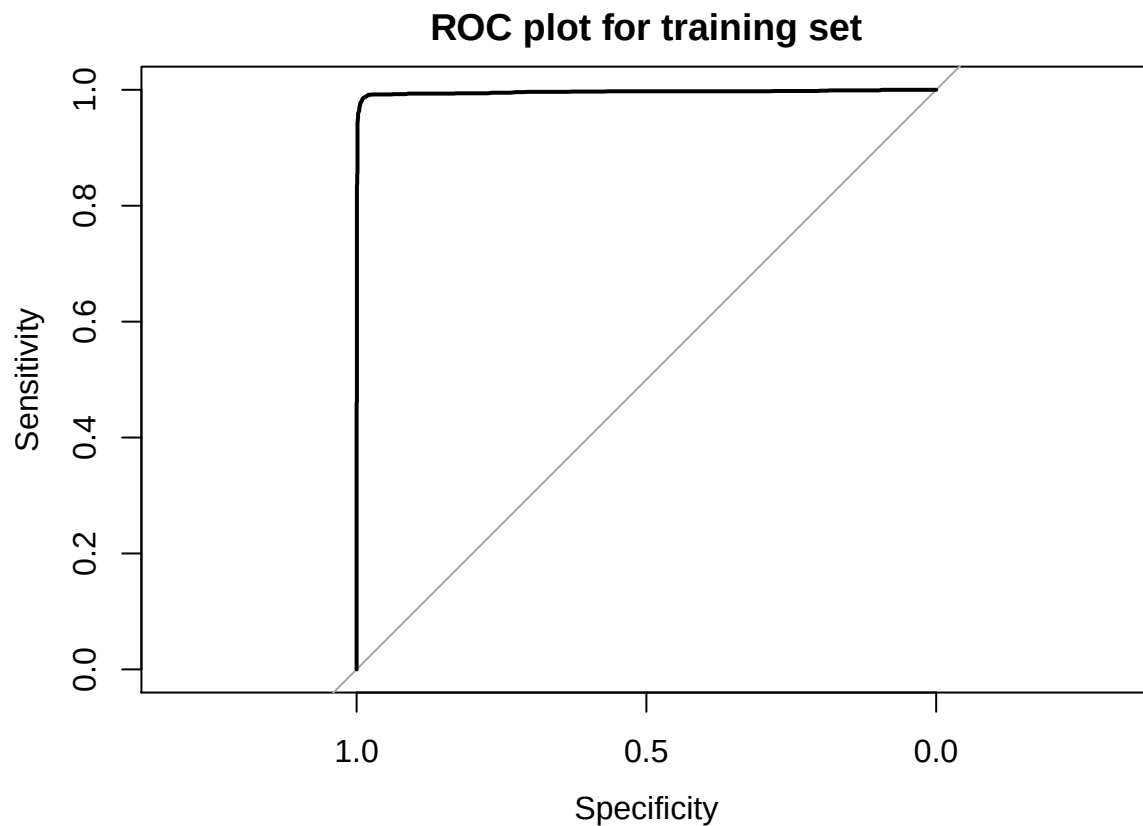
```
## Setting levels: control = 0, case = 1
```

```
## Setting direction: controls < cases
```

```
auc(mailing_train_roc)
```

```
## Area under the curve: 0.996
```

```
plot(mailing_train_roc, main = "ROC plot for training set")
```



Now create a ROC chart for the test set and compute the test AUC.

```
g <-predict(rf, newdata = mailing_test, type = "prob") [,2] %>%  
  as.numeric  
mailing_test_roc <- roc(mailing_test$class, g)
```

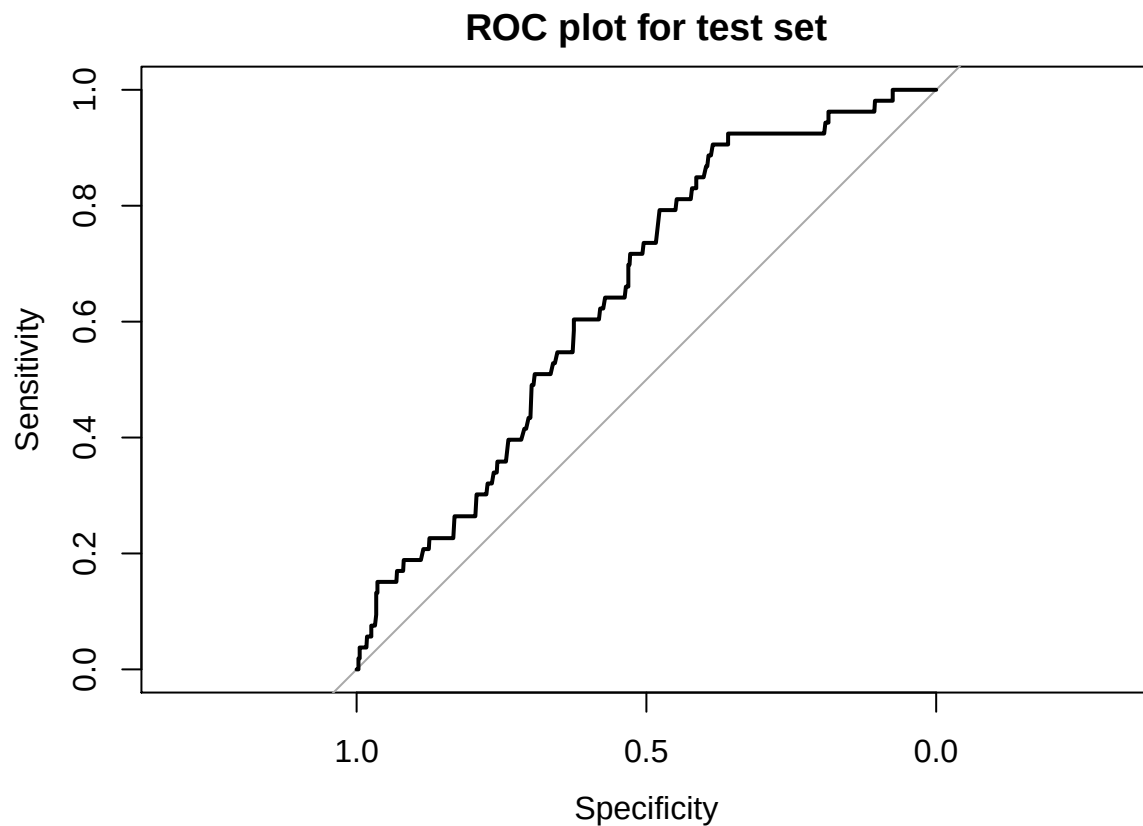
```
## Setting levels: control = 0, case = 1
```

```
## Setting direction: controls < cases
```

```
auc(mailing_test_roc)
```

```
## Area under the curve: 0.6504
```

```
plot(mailing_test_roc, main = "ROC plot for test set")
```



>I decide to combine the ROC plot for both the training set and the test set to make it easier to see the difference.

```
auc_plot <- plot(roc(mailing_train$class, p), print.auc = TRUE, col = "green")
```

```
## Setting levels: control = 0, case = 1
```

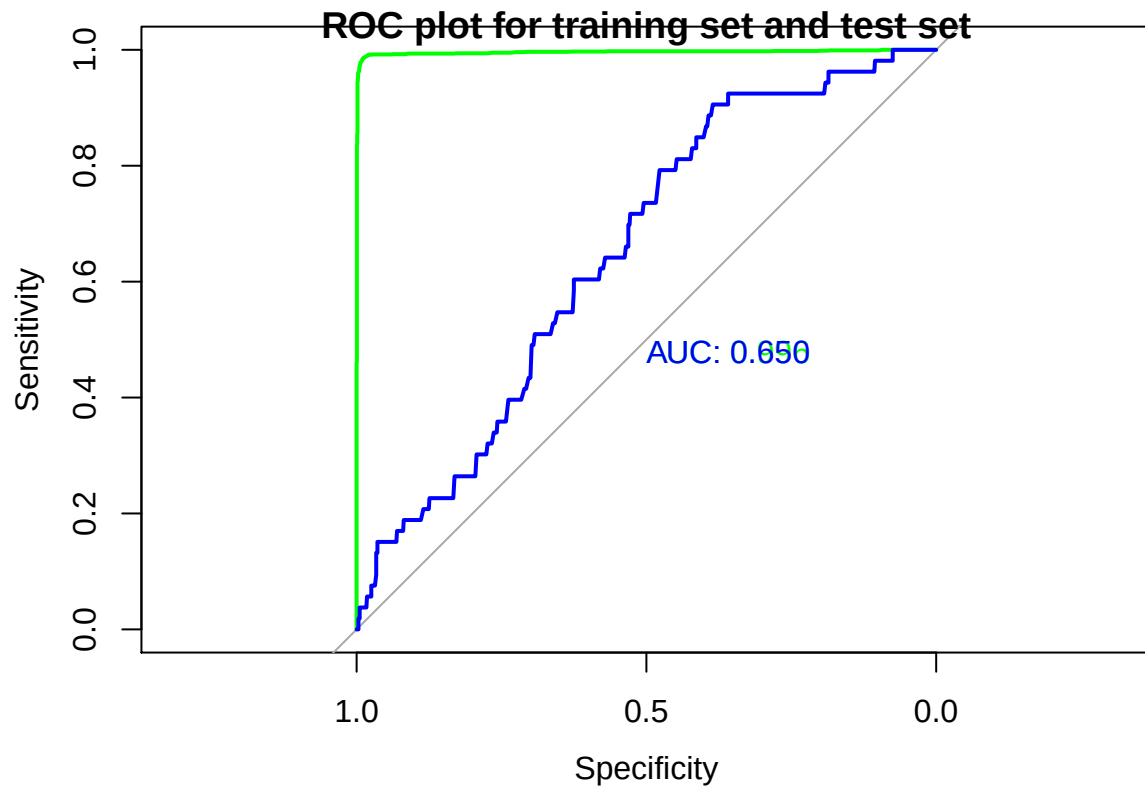
```
## Setting direction: controls < cases
```

```
auc_plot <- plot(roc(mailing_test$class, g), print.auc = TRUE, col = "blue", add = TRUE)
```

```
## Setting levels: control = 0, case = 1
```

```
## Setting direction: controls < cases
```

```
auc_plot <- title(main = "ROC plot for training set and test set")
```

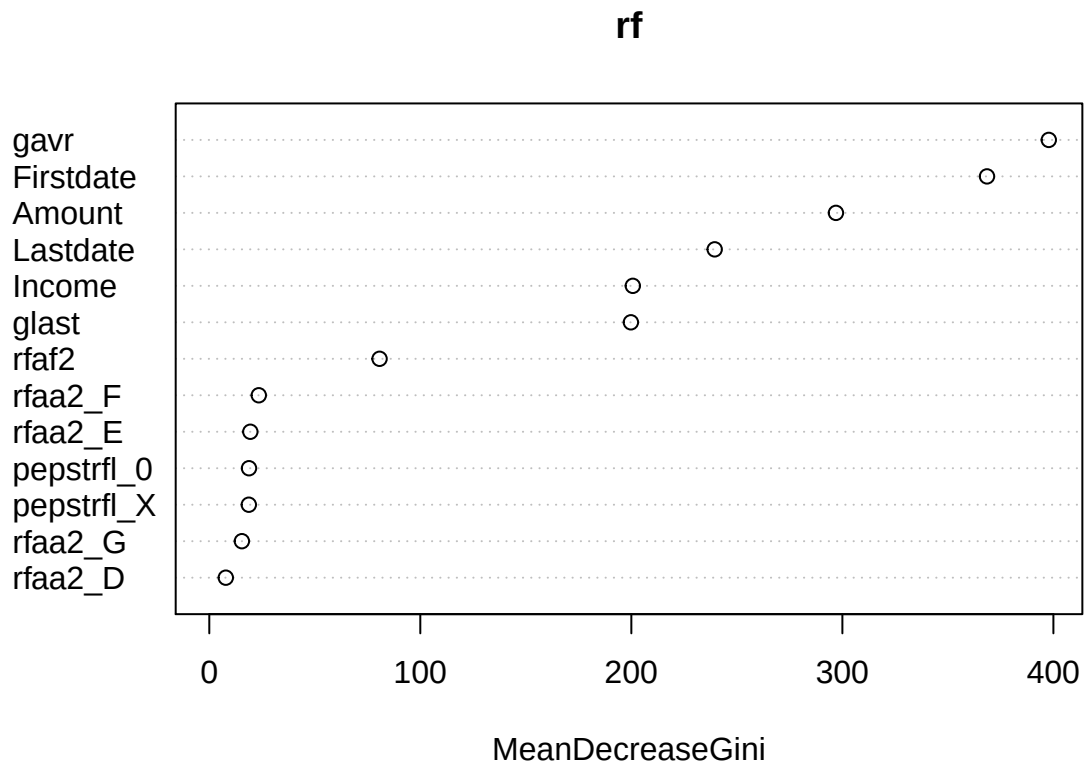


Is the model underfit, overfit, or correctly fit to the data? > The AUC of mailing_train has a excellent value of 99.60%, while the AUC of mailing_test has an acceptable value of 65.04%. Comparing the gaps between the test and training set, the difference between mailing_train and mailing_test is 34.56%, which suggest that the model over-fits the data.

3. Examine the model

Examine the variable importance of the model.

```
varImpPlot(rf)
```



>The most important variable is ‘gavr’, which indicates the amount of average gift that a individual gives.

Make some individual predictions of the model. Choose one case from the data and see what the model predicts for that one person.

```
d <- mailing_train[20,]
predict(rf, newdata = d, type = "prob")
```

```
##      0      1
## 1 0.115 0.885
## attr("class")
## [1] "matrix" "array"  "votes"
```

Using the *most important* variable from the plot above, change the value of that variable to something new and make a new predicting for that one case (i.e. set the value to something very small, or very large). How does the prediction change?

```
d$gavr <- 20
predict(rf, newdata = d, type = "prob")
```

```
##      0      1
## 1 0.302 0.698
## attr("class")
## [1] "matrix" "array"  "votes"
```

```
d$gavr <- 80
predict(rf, newdata = d, type = "prob")
```

```
##      0      1
## 1 0.36 0.64
## attr(,"class")
## [1] "matrix" "array"  "votes"
```

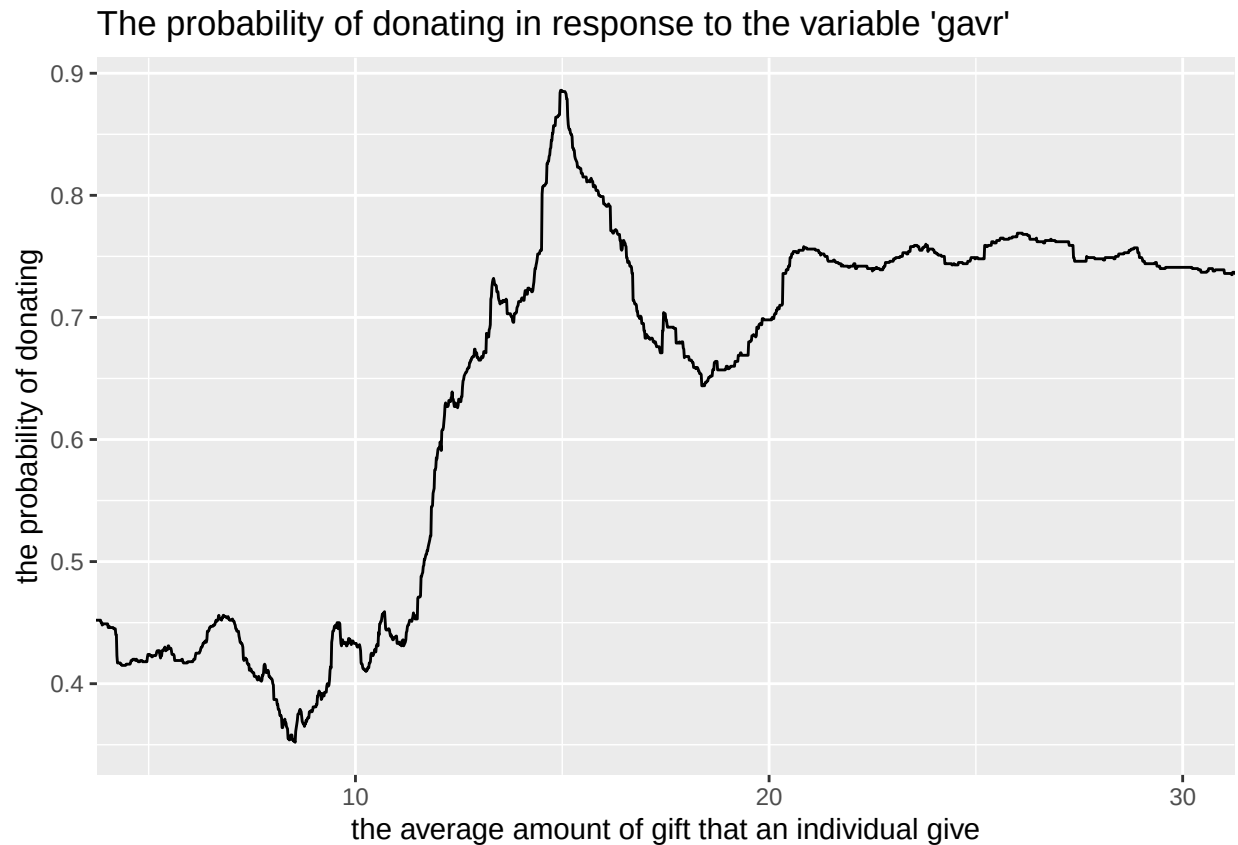
If I set the value to 20, there is a 69.8% chance that the individual will donate. If I set the value to 80, there is a 64% chance that the individual will donate. Thus, we can say that this individual is likely to donate in all cases

Let's now try and predict the outcome for this case if that important variable was changed from it's minimum value to it's maximum value. 1. Create a grid of at least 100 points from the minimum value to the maximum value. 2. Duplicate the case you used above the same number of times 3. Add the grid of points to the data 4. Predict the outcome using this new fake data and save the predicted probability in the dataset.

```
N <- 10000
L <- seq(min(mailing_train$gavr),
        max(mailing_train$gavr),
        length.out = N)
dn <- d %>%
  mutate(n = N) %>%
  uncount(n) %>%
  mutate(gavr = L)
dn$pred <- predict(rf, newdata = dn, type = "prob") [,2]
```

Now plot the results of that sequential grid against the predicted probability. How do we see the probability of responding the mailer change in response to the variable? Tip: You can use `coord_cartesian` to change the xlimits to focus on specific areas of detail if you want.

```
ggplot(dn, aes(x = gavr, y = pred)) +
  geom_line() +
  coord_cartesian(xlim = c(5,30)) +
  labs(title = "The probability of donating in response to the variable 'gavr'") +
  xlab("the average amount of gift that an individual give") +
  ylab("the probability of donating")
```



Bonus: Use IML to create a PDP+ICE chart

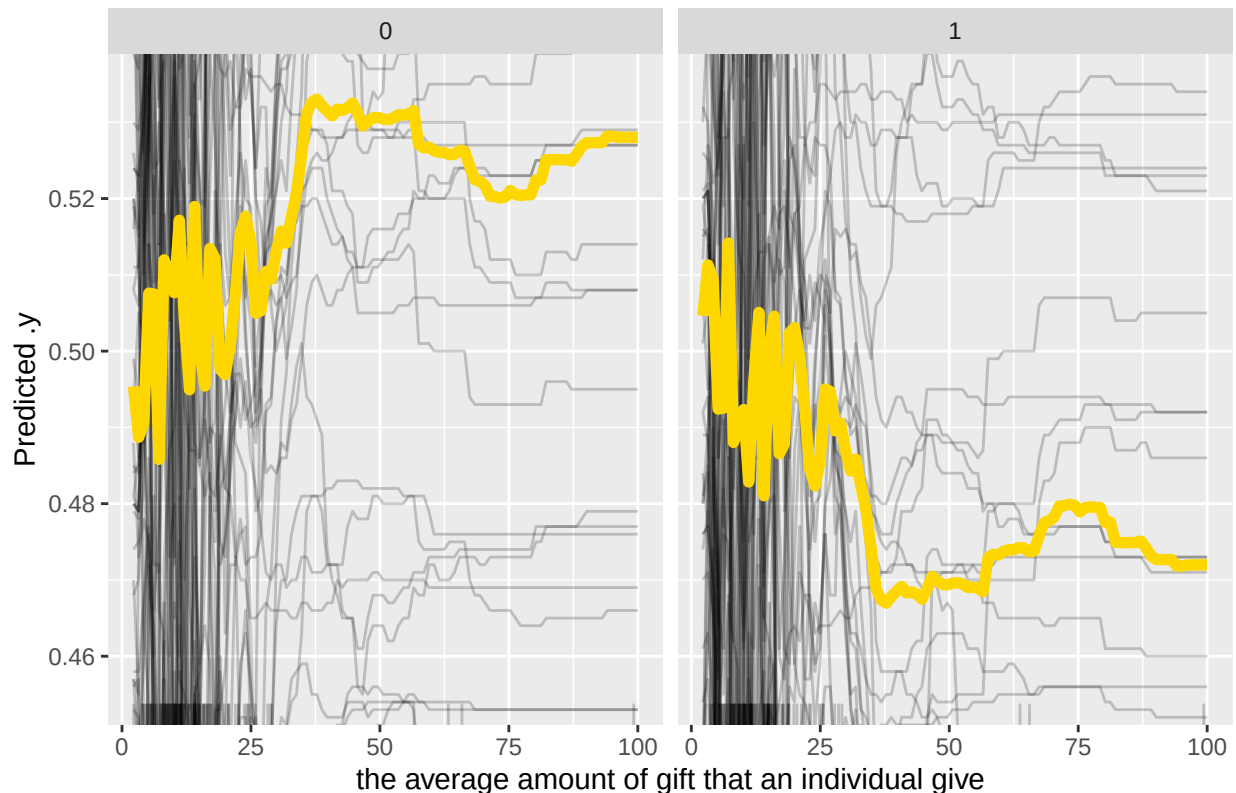
Use the `iml` package to create an individual conditional expectations combined with partial dependence chart. It's highly recommended that you only provide the prediction object a subset of the data. I'm not sure the RStudio Cloud instance can take the full data and it will take a very long time. Use `coord_cartesian` to focus the y-limit range to focus the chart to a region where you can observe the effect (it's tiny!).

```
library(iml)
A <- sample_n(mailing_train, 110)
mailing_pred <- Predictor$new(model = rf, data = A, y = A$class, type = "prob")

mailing_effect <- FeatureEffect$new(mailing_pred, grid.size = 100, method = "pdp+ice", feature = "gavr")

mailing_effect$plot() +
  coord_cartesian(ylim = c(0.455, 0.535)) +
  xlab("the average amount of gift that an individual give") +
  ylab("the probability of donating") +
  labs(title = "The probability of donating in response to the variable 'gavr'")
```

The probability of donating in response to the variable 'gavr



4. Summarize

What are your thoughts on the impact of these different features on the likelihood for a person to respond to our donation requests? (100-200 words)

Things you could write about:

- What changes would you make to the modeling approach to improve the predictions?
- What recommendations would you make to the organization?
- What other kinds of data would be useful in improving these predictions?

One changes I would make to the modeling approach to introduce a 'Four-fold cross validation' to improve prediction. What this method does is to divided the training data into 4 blocks. I will select 1 block for the validation and 3 blocks for testing. This action will be repeated when every single block is validated. By completing the 'four-fold cross validation', we can further confirm whether the data is overfitted and make suitable adjustments to the data.

Base on the model that shows the average amount of gift that an individual give and the probability of donating, suggest that if the individual donates on average the value of 15 or above, there is a chance of more than 50% of the individual would donate

Text analytics on email response would be a useful data in improving these predic-tions. For example, data analyst can use term frequency to helps to identify and

categorize individuals base on common terminologies that they type in an email and predict whether the individuals will donate.